

CCPROG3 MCO2: Object-Oriented Design and Implementation of a Java Application

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
IngredientType.isRamenIngredient	1	A noodle is usable in a ramen	NOODLE	true	true	P
	2	A broth is usable in a ramen	BROTH	true	true	P
	3	An unrelated item is not usable in a ramen	NONE	false	false	P
AlertType.isCritical	1	An empty slot is critical	SLOT_EMPTY	true	true	P
	2	Low stock is only a warning	SLOT_LOW	false	false	P
	3	A low change fund is critical	CHANGE_FUND_LOW	true	true	P
Alert.isCritical	1	Reports the severity of a critical type	type=SLOT_EMPTY	true	true	P
	2	Reports the severity of a warning type	type=SLOT_LOW	false	false	P
	3	Severity follows the type, not the figures	type=DENOMINATION_LOW	false	false	P
Item.toString	1	Describes a normal item	Gyoza, 260 cal	Gyoza (260 cal)	Gyoza (260 cal)	P
	2	Describes a zero-calorie item	Bottled Water, 0 cal	Bottled Water (0 cal)	Bottled Water (0 cal)	P
	3	Describes an item with a long name	Black Garlic Oil, 40 cal	Black Garlic Oil (40 cal)	Black Garlic Oil (40 cal)	P
Money.addQty	1	Adds pieces to an empty tray	qty=0, add 5	5	5	P
	2	Adds further pieces	qty=5, add 3	8	8	P
	3	Ignores a negative amount	qty=8, add -2	8	8	P
Money.removeQty	1	Removes some pieces	qty=8, remove 3	true	true	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	2	Leaves the correct remainder	after removing 3 of 8	5	5	P
	3	Refuses to remove more than is held	qty=5, remove 9	false	false	P
Money.getTotalValue	1	Values an empty tray	value=50, qty=0	0	0	P
	2	Values a stocked tray	value=50, qty=4	200	200	P
	3	Values a single piece	value=1000, qty=1	1000	1000	P
ItemSlot.stock(Item)	1	Accepts a unit into an empty slot	capacity=10, add 1 Item	true	true	P
	2	Stock rises by one	after adding 1 unit	1	1	P
	3	Refuses a unit when the slot is full	capacity=2, holding 2	false	false	P
ItemSlot.stock(int)	1	Creates the requested number of units	capacity=10, stock 6	true	true	P
	2	Holds exactly that many units	after stocking 6	6	6	P
	3	Refuses a quantity beyond capacity	holding 6 of 10, stock 5	false	false	P
ItemSlot.dispense	1	Returns one unit of the right item	stock=3, dispense	Gyoza	Gyoza	P
	2	Records the sale and the revenue	after one sale at Php 50	qty=2 sold=1 revenue=50	qty=2 sold=1 revenue=50	P
	3	Returns nothing when the slot is empty	stock=0, dispense	null	null	P
ItemSlot.consume	1	Returns the units taken	stock=5, consume 2	2	2	P
	2	Counts them as ingredients, not as sales	after consuming 2 at Php 25	qty=3 sold=0 used=2 revenue=0	qty=3 sold=0 used=2 revenue=0	P
	3	Refuses to take more than is held	stock=3, consume 4	null	null	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
ItemSlot.resetSummary	1	Starting stock becomes the current stock	stock=7 after activity	7	7	P
	2	Both usage counters return to zero	after reset	sold=0 used=0	sold=0 used=0	P
	3	Revenue returns to zero	after reset	0	0	P
ItemSlot.isEmpty	1	A new slot is empty	stock=0	true	true	P
	2	A stocked slot is not empty	stock=1	false	false	P
	3	A drained slot is empty again	stock=1, dispense 1	true	true	P
ItemSlot.isFull	1	A new slot is not full	capacity=3, stock=0	false	false	P
	2	A partly stocked slot is not full	capacity=3, stock=2	false	false	P
	3	A slot at capacity is full	capacity=3, stock=3	true	true	P
RamenPreset.addIngredient	1	Accepts a valid component	Noodles x1	1	1	P
	2	Keeps components in order with quantities	then Tonkotsu Broth x2	Tonkotsu Broth=2	Tonkotsu Broth=2	P
	3	Ignores a component with no orders	Negi x0	2	2	P
RamenPreset.size	1	An empty recipe has no components	new preset	0	0	P
	2	Counts every component added	3 components added	2	2	P
	3	A null component is not counted	addIngredient(null, 2)	0	0	P
VendingMachine.addSlot	1	Adds a brand new item	S01 Gyoza, Php 50, qty 6	true	true	P
	2	Refuses a duplicate slot code	S01 again with a new name	false	false	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	3	Refuses a duplicate item name	S02 Gyoza	false	false	P
	4	Refuses a quantity beyond slot capacity	S03 qty 11	false	false	P
	5	Refuses a blank item name	S04 name=""	false	false	P
	6	Refuses a negative price	S05 price=-1	false	false	P
VendingMachine.restock	1	Adds stock to an existing slot	slot 1 at 9 of 10, add 1	true	true	P
	2	Stock reaches the new level	after restocking	10	10	P
	3	Refuses to exceed slot capacity	slot 1 at 10 of 10, add 1	false	false	P
	4	Refuses an unknown slot number	slot 99	false	false	P
VendingMachine.setItemPrice	1	Sets a new price	slot 1 to Php 30	true	true	P
	2	The slot reports the new price	after repricing	30	30	P
	3	Accepts a price of zero	slot 2 to Php 0	true	true	P
	4	Refuses a negative price	slot 1 to Php -5	false	false	P
	5	Refuses an unknown slot number	slot 99 to Php 30	false	false	P
VendingMachine.removeSlot	1	Removes an existing slot	slot 1 of 10	true	true	P
	2	The machine holds one slot fewer	after removal	9	9	P
	3	Refuses an unknown slot number	slot 99	false	false	P
VendingMachine.findSlotByCode	1	Finds a slot by its exact code	S01	Noodles	Noodles	P
	2	Ignores capitalisation	s01	Noodles	Noodles	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	3	Returns nothing for an unknown code	S99	null	null	P
VendingMachine.findSlotByName	1	Finds a slot by its item name	Chashu Pork	S02	S02	P
	2	Ignores capitalisation	chashu pork	S02	S02	P
	3	Returns nothing for an unstocked item	Sushi	null	null	P
VendingMachine.findSlotNumberByName	1	Reports the first slot as one, not zero	Noodles	1	1	P
	2	Reports a later slot correctly	Gyoza	6	6	P
	3	Reports -1 for an unstocked item	Sushi	-1	-1	P
VendingMachine.getSlot	1	Slot numbering starts at one	slot 1	S01	S01	P
	2	Returns the last slot	slot 10	S10	S10	P
	3	Rejects zero as a slot number	slot 0	null	null	P
	4	Rejects a number past the last slot	slot 11	null	null	P
VendingMachine.meetsMinimumSlots	1	A default machine meets the 8-slot minimum	10 slots	true	true	P
	2	An empty machine does not	0 slots	false	false	P
	3	A machine trimmed below eight does not	7 slots	false	false	P
VendingMachine.replenishMoney(int)	1	Adds a single piece	1 peso	true	true	P
	2	The tray holds one piece	after adding one	1	1	P
	3	Refuses an unsupported face value	3 peso	false	false	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
VendingMachine.replenishMoney(int, int)	1	Adds several pieces at once	20 peso x 15	true	true	P
	2	The tray holds them all	after adding 15	15	15	P
	3	Refuses a quantity of zero	20 peso x 0	false	false	P
VendingMachine.collectMoney	1	Withdraws pieces that are held	20 peso x 5 of 15	true	true	P
	2	The tray falls to the right level	after withdrawing 5	10	10	P
	3	Refuses to withdraw more than is held	20 peso x 99	false	false	P
VendingMachine.getMoneyTotal	1	A new machine holds no money	empty fund	0	0	P
	2	Values a single denomination	100 x 3	300	300	P
	3	Values several denominations	100 x 3 plus 20 x 4	380	380	P
VendingMachine.getCashTotal	1	Values an empty insertion	all zeroes	0	0	P
	2	Values a single note	one 100	100	100	P
	3	Values a mixed insertion	one 100, two 20, one 5	145	145	P
VendingMachine.findDenominationIndex	1	Finds the smallest denomination	1 peso	0	0	P
	2	Finds the largest denomination	1000 peso	8	8	P
	3	Reports -1 for an unsupported value	3 peso	-1	-1	P
VendingMachine.makeChangePlan	1	Finds a plan by backtracking, not greedily	owe 75 from 50+20+5	75	75	P
	2	Reports failure when no combination fits	owe 76 from 50+20+5	null	null	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	3	Zero change always succeeds	owe 0	0	0	P
	4	Can pay change out of the cash just inserted	owe 20, fund empty, inserted 20	20	20	P
VendingMachine.purchaseItem	1	Completes a sale and gives change	slot 1 at Php 25, insert 100	true SUCCESS change=75	true SUCCESS change=75	P
	2	Stock falls by one	after the sale	9	9	P
	3	Refuses when too little cash is inserted	slot 1 at Php 25, insert 10	INSUFFICIENT_PAYMENT	INSUFFICIENT_PAYMENT	P
	4	Refuses when exact change cannot be made	insert 100, empty fund	NO_EXACT_CHANGE	NO_EXACT_CHANGE	P
	5	A refused sale leaves the stock untouched	after the refusal	10	10	P
	6	Refuses when the slot is empty	slot 1 drained	OUT_OF_STOCK	OUT_OF_STOCK	P
	7	Refuses an item that is not sold individually	slot 7 Tonkotsu Broth	NOT_SOLD_INDIVIDUALLY	NOT_SOLD_INDIVIDUALLY	P
	8	Refuses an unknown slot number	slot 99	INVALID_SLOT	INVALID_SLOT	P
VendingMachine.resetTransactionSummary	1	Starting stock matches current stock	after a sale then a reset	9	9	P
	2	Sales counters return to zero	after the reset	0	0	P
	3	Total sales return to zero	after the reset	0	0	P
VendingMachine.getTotalSales	1	A fresh machine has taken nothing	no sales	0	0	P
	2	Counts a single sale	one Noodles at Php 25	25	25	P
	3	Adds a second sale	plus one Chashu Pork at Php 45	70	70	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
VendingMachine.getAlerts	1	Flags an empty change fund on a new machine	fund = Php 0	true	true	P
	2	Reports nothing on a fully stocked, funded machine	10 slots stocked, fund Php 480	0	0	P
	3	Warns when a slot runs low	Noodles down to 2 of 10	SLOT_LOW:Noodles	SLOT_LOW:Noodles	P
	4	Raises a critical alert for an empty slot	Noodles at 0	true	true	P
	5	Flags a machine with too few slots	empty machine, 0 slots	true	true	P
RegularVendingMachine.getMachineName	1	Names itself as a regular machine	new machine	Regular Vending Machine	Regular Vending Machine	P
	2	The name does not change once stocked	after loadDefaultItems	Regular Vending Machine	Regular Vending Machine	P
	3	Every instance reports the same name	second instance	Regular Vending Machine	Regular Vending Machine	P
RegularVendingMachine.loadDefaultItems	1	Stocks the ten default items	loadDefaultItems on a new machine	10	10	P
	2	Every default item can be sold on its own	inspect all 10 slots	true	true	P
	3	No slot is stocked beyond capacity	inspect all 10 slots	0	0	P
	4	Loading twice does not duplicate slots	loadDefaultItems called again	10	10	P
SpecialVendingMachine.getMachineName	1	Names itself as a special machine	new machine	Special Ramen Vending Machine	Special Ramen Vending Machine	P
	2	Differs from the regular machine's name	compare the two	true	true	P
	3	The name does not change once stocked	after loadDefaultItems	Special Ramen Vending Machine	Special Ramen Vending Machine	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
SpecialVendingMachine.loadDefaultItems	1	Stocks the thirteen default items	loadDefaultItems	13	13	P
	2	Broth cannot be sold on its own	slot 7 Tonkotsu Broth	false	false	P
	3	Loads the ready-made recipes alongside the stock	after loadDefaultItems	4	4	P
SpecialVendingMachine.getIngredientSlots	1	Lists only slots usable in a ramen	13 slots, 3 unrelated	10	10	P
	2	Every listed slot really is an ingredient	inspect the list	true	true	P
	3	Excludes items marked as unrelated	Gyoza is NONE	false	false	P
SpecialVendingMachine.getTotalSales	1	Counts an individual sale	one Noodles at Php 25	25	25	P
	2	Adds ramen revenue on top	plus one ramen at Php 60	85	85	P
	3	Reports the ramen revenue separately	after the ramen	60	60	P
SpecialVendingMachine.resetTransactionSummary	1	Clears the ramen revenue	after a reset	0	0	P
	2	Clears the ramen count	after a reset	0	0	P
	3	Clears the item revenue too	after a reset	0	0	P
SpecialVendingMachine.prepareRamen	1	Assembles a valid bowl	noodles, 2 chashu, tonkotsu; insert 200	true SUCCESS	true SUCCESS	P
	2	Ingredients are consumed, not sold	chashu after using 2	qty=6 sold=0 used=2	qty=6 sold=0 used=2	P
	3	Refuses a bowl with no broth	noodles and chashu only	MISSING_NOODLES_OR_BROTH	MISSING_NOODLES_OR_BROTH	P
	4	Refuses a bowl with no noodles	chashu and tonkotsu only	MISSING_NOODLES_OR_BROTH	MISSING_NOODLES_OR_BROTH	P
	5	Refuses an item that is not an ingredient	includes Gyoza	NOT_A_RAMEN_INGREDIENT	NOT_A_RAMEN_INGREDIENT	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	6	Refuses when too little cash is inserted	insert 5 for a Php 60 bowl	INSUFFICIENT_PAYMENT	INSUFFICIENT_PAYMENT	P
	7	Refuses when an ingredient is short	11 noodles from a stock of 10	OUT_OF_STOCK	OUT_OF_STOCK	P
	8	Refuses an empty order	no ingredients chosen	INVALID_ORDER	INVALID_ORDER	P
SpecialVendingMachine.computeRamenCalories	1	Adds a broth-absorption bonus to the noodles	1 noodles (200) + 1 tonkotsu (150)	370	370	P
	2	Raises the whole total when an add-on is present	2 noodles, 1 chashu, 1 tonkotsu, 1 garlic oil	977	977	P
	3	Is a plain sum when neither applies	1 chashu (300) + 1 negi (10)	310	310	P
	4	Scales with the number of orders	2 chashu (300 each)	600	600	P
SpecialVendingMachine.computeRamenPrice	1	Prices a simple bowl	noodles 25 + tonkotsu 35	60	60	P
	2	Multiplies by the number of orders	2 chashu at 45	90	90	P
	3	Follows the operator's current prices	noodles repriced to 30	65	65	P
SpecialVendingMachine.getLastPreparation	1	Reports nothing before any ramen is made	new machine	0	0	P
	2	Narrates every stage of a full bowl	noodles, chashu, broth, oil	7	7	P
	3	Starts by blanching the noodles	first step	true	true	P
	4	Ends by announcing the finished bowl	last step	Custom Ramen done!	Custom Ramen done!	P
SpecialVendingMachine.loadDefaultPresets	1	Loads the four recipes	after loadDefaultItems	4	4	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	2	Names the first recipe	preset 1	Tonkotsu Classic	Tonkotsu Classic	P
	3	Loading twice does not duplicate recipes	called again	4	4	P
SpecialVendingMachine.isPresetAvailable	1	A recipe is available on a fully stocked machine	Tonkotsu Classic	true	true	P
	2	Becomes unavailable once an ingredient runs out	noodles drained to 0	false	false	P
	3	An empty recipe is never available	preset with no components	false	false	P
SpecialVendingMachine.getPresetPrice	1	Prices a recipe at the machine's current prices	1 noodles 25, 2 chashu 45, 1 egg 15, 1 tonkotsu 35	165	165	P
	2	Follows a price change	noodles repriced 25 to 40	180	180	P
	3	Prices an unknown recipe at zero	component not stocked	0	0	P
UI.denominationLabel	1	Labels a small value as a coin	1	1 Peso Coin	1 Peso Coin	P
	2	Labels the boundary value as a coin	20	20 Peso Coin	20 Peso Coin	P
	3	Labels a large value as a bill	100	100 Peso Bill	100 Peso Bill	P
UI.table	1	Builds a table with the given headings	3 columns	3	3	P
	2	Starts with no rows	new table	0	0	P
	3	Cells cannot be edited by the user	row 0, column 0	false	false	P
UI.setRows	1	Fills the table with the rows given	2 rows	2	2	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	2	Keeps the values in place	row 1, column 2	6	6	P
	3	Replaces rather than appends	1 row after 2	1	1	P
MainFrame.showCard	1	Shows the factory menu	FACTORY	true	true	P
	2	Switches to the vending screen	VENDING	true	true	P
	3	Hides the screen it switched away from	after VENDING	false	false	P
	4	Switches to the maintenance screen	MAINTENANCE	true	true	P
MainFrame.showDialog	1	Confirms a machine has been created	Create a Regular Vending Machine	dialog names the machine		P
	2	Warns that an empty machine has no items	untick defaults, then create	dialog mentions adding items		P
	3	Reminds the operator to add money	create any machine	dialog mentions replenishing money		p
VendingPanel.setItemRows	1	Nothing is selected until the user chooses	1 row loaded	-1	-1	P
	2	An empty machine leaves the table empty	0 rows	-1	-1	P
	3	Accepts a running total without error	Php 120	true	true	P
VendingPanel.appendOutput	1	Writes a line to the output area	one line appended	true	true	P
	2	Keeps earlier lines when appending	second line appended	true	true	P
	3	setOutput replaces rather than appends	setOutput after two lines	false	false	P
	4	Names the coin that was accepted	insert a 20 peso coin	true	true	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	5	States the new running total	after inserting 20	true	true	P
	6	Adds to the output instead of replacing it	append a second line	true	true	P
VendingPanel.setRamenButtonVisible	1	Shows the button when asked	true	true	true	P
	2	Hides the button when asked	false	false	false	P
	3	A regular machine hides it automatically	setMachine(regular)	false	false	P
	4	A special machine shows it automatically	setMachine(special)	true	true	P
	5	Shows the button on a special machine	special machine	true	true	P
	6	Hides it on a regular machine	regular machine	false	false	P
	7	Follows the machine currently under test	switch to a regular machine	false	false	P
RamenPanel.setPresetOptions	1	Selects the first recipe by default	3 recipes	0	0	P
	2	Replaces the list rather than adding to it	1 recipe	0	0	P
	3	Reports no selection when the list is empty	0 recipes	-1	-1	P
RamenPanel.getRequestQuantity	1	Defaults to a single order	untouched spinner	1	1	P
	2	Is unaffected by the bowl contents	after loading a bowl	1	1	P
	3	Never returns zero	untouched spinner	true	true	P
RamenPanel.setTotals	1	Shows the running price	price 140	true	true	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	2	Shows the running calorie count	calories 977	true	true	P
	3	Returns to zero for an empty bowl	price 0, calories 0	true	true	P
	4	Starts an empty bowl at nothing	cleared bowl	Price: Php 0 Calories: 0	Price: Php 0 Calories: 0	P
	5	Shows the price and calories of a bowl	noodles and tonkotsu	Price: Php 60 Calories: 370	Price: Php 60 Calories: 370	P
	6	Returns to zero when the bowl is cleared	after clearing	Price: Php 0 Calories: 0	Price: Php 0 Calories: 0	P
RamenPanel.setControlsEnabled	1	Locks the controls	false	false	false	P
	2	Unlocks the controls	true	true	true	P
	3	Can be locked and unlocked repeatedly	false then true	true	true	P
	4	Locks the controls during preparation	narration running	false	false	P
	5	Unlocks them once the bowl is done	narration finished	true	true	P
	6	Setting it twice is harmless	already unlocked	true	true	P
RamenPanel.appendOutput	1	Writes a preparation step	one step appended	true	true	P
	2	Steps accumulate in order	two steps appended	true	true	P
	3	setOutput clears the area	setOutput("")	true	true	P
	4	Shows every preparation step	a full bowl	7	7	P
	5	Opens by blanching the noodles	first line	true	true	P
	6	Closes by announcing the bowl	last line	true	true	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
RamenPanel.setBowlRows	1	Lists every component chosen	2 components	2	2	P
	2	An emptied bowl lists nothing	0 components	0	0	P
	3	Replaces rather than appends	1 component after 0	1	1	P
MaintenancePanel.clearForm	1	Leaves the item name blank	after clearForm	empty	empty	P
	2	Leaves the price blank	after clearForm	empty	empty	P
	3	Leaves the quantity and calories blank	after clearForm	empty	empty	P
MaintenancePanel.getSelectedDenomination	1	Starts on the smallest denomination	untouched	0	0	P
	2	Is a valid index into the denominations	untouched	true	true	P
	3	Reports a positive default piece count	untouched spinner	true	true	P
MaintenancePanel.setSummaryRows	1	Lists one row per slot	13 slots	13	13	P
	2	Separates items sold from items used in a ramen	Noodles: 1 sold, 1 used	1 1	1 1	P
	3	Starting matches ending after a reset	reset then refresh	true	true	P
	4	Shows one row per slot	13 slots	13	13	P
	5	Separates items sold from items used in a ramen	one Noodles sold, one used	1 1	1 1	P
	6	Both counters clear on a new summary period	after reset	0 0	0 0	P
MaintenancePanel.setAlertRows	1	Reports nothing on a healthy machine	stocked and funded	0	0	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	2	Marks an empty slot as critical	Noodles drained	CRITICAL	CRITICAL	P
	3	Marks low stock as a warning only	Noodles down to 2	Warning	Warning	P
	4	Shows nothing on a healthy machine	stocked and funded	0	0	P
	5	Reports an emptied slot as critical	Noodles drained	CRITICAL	CRITICAL	P
	6	Names the item the alert concerns	Noodles drained	Noodles	Noodles	P
MaintenancePanel.showMessage	1	Writes a line of feedback	one message	true	true	P
	2	Keeps earlier feedback	second message	true	true	P
	3	Messages appear in the order they were written	two messages	true	true	P
	4	Confirms a successful restock	restock slot 1	true	true	P
	5	Explains a refused restock	restock beyond capacity	true	true	P
	6	Keeps earlier messages on screen	two messages shown	true	true	P
FactoryController.hasCurrentMachine	1	Reports no machine before one is built	new controller	false	false	P
	2	Reports a machine once one is built	after createRegularMachine	true	true	P
	3	Still reports one after building a second	after createSpecialMachine	true	true	P
FactoryController.createRegularMachine()	1	Builds a regular machine	no arguments	Regular Vending Machine	Regular Vending Machine	P
	2	Leaves it unstocked	no arguments	0	0	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	3	Replaces any previous machine	called twice	true	true	P
FactoryController.createRegularMachine(boolean)	1	Stocks the machine when asked	loadDefaults = true	10	10	P
	2	Leaves it empty when not asked	loadDefaults = false	0	0	P
	3	Builds the right type either way	loadDefaults = false	Regular Vending Machine	Regular Vending Machine	P
FactoryController.createSpecialMachine(boolean)	1	Builds a special machine	loadDefaults = true	Special Ramen Vending Machine	Special Ramen Vending Machine	P
	2	Stocks the thirteen default items	loadDefaults = true	13	13	P
	3	Leaves it empty with no arguments	no arguments	0	0	P
VendingController.insertMoney	1	Accepts a supported denomination	100	true	true	P
	2	The running total reflects it	after inserting 100	100	100	P
	3	Refuses an unsupported denomination	3	false	false	P
VendingController.getInsertedTotal	1	Adds a second piece to the total	100 then 20	120	120	P
	2	Adds a coin to the total	plus 1	121	121	P
	3	Counts only what has been inserted	no further insertions	121	121	P
VendingController.refundMoney	1	Returns everything inserted	121 inserted	121	121	P
	2	The tally is emptied	after the refund	0	0	P
	3	A second refund returns nothing	already refunded	0	0	P
VendingController.buyItem	1	Completes a valid purchase	slot 1 at Php 25, inserted 100	true	true	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	2	The inserted cash is cleared afterwards	after the sale	0	0	P
	3	Refuses when too little is inserted	slot 1, inserted 1	false	false	P
	4	The cash stays inserted after a refusal	after the refusal	1	1	P
VendingController.buyCustomRamen	1	Assembles and sells a valid bowl	noodles and tonkotsu, inserted 200	true	true	P
	2	Refuses a bowl with no broth	noodles only	false	false	P
	3	Refuses a ramen on a regular machine	regular machine	false	false	P
VendingController.narrate	1	Steps appear one at a time, not all at once	press Prepare and buy	seven lines about 0.7s apart	7 lines appeared in sequence, around 5s total	P
	2	The controls stay locked until the last step	watch during the narration	buttons greyed, then usable	buttons greyed during narration, usable after	P
	3	The change is reported after the final step	insert P200 for a P140 bowl	Php 60 listed after "done!"	Php 60 change listed after the final line	P
MaintenanceController.restock	1	Restocks an existing slot	slot 1 at 9 of 10, add 1	true	true	P
	2	Refuses to exceed capacity	slot 1 at 10 of 10, add 1	false	false	P
	3	Refuses an unknown slot	slot 99	false	false	P
MaintenanceController.setPrice	1	Sets a new price	slot 1 to Php 30	true	true	P
	2	Refuses a negative price	slot 1 to Php -1	false	false	P
	3	Refuses an unknown slot	slot 99 to Php 30	false	false	P
MaintenanceController.replenishMoney	1	Adds pieces of a denomination	50 peso x 4	true	true	P
	2	The fund reflects the addition	after adding 4 x 50	200	200	P

Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
	3	Refuses an unsupported denomination	3 peso x 4	false	false	P
MaintenanceController.collectMoney	1	Withdraws pieces that are held	50 peso x 2 of 4	true	true	P
	2	The fund falls accordingly	after withdrawing 2 x 50	100	100	P
	3	Refuses to withdraw more than is held	50 peso x 99	false	false	P
Driver.main	1	Opens the window on start-up	run java Driver	window appears, no console error	window opened, no console errors	P
	2	Closing the window ends the program	close the window	process terminates	process terminated	P
	3	The Exit button ends the program	press Exit on the factory menu	process terminates	process terminated	P